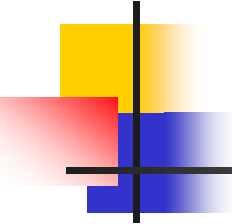


算法设计与分析

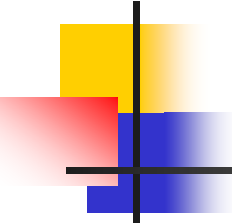
Analysis and Design of Algorithm

Lesson 08



要点回顾

- 动态规划算法的设计要素
 - **建模**：目标函数、约束条件
 - **分段**：确定子问题的**边界**
 - **分析**：子问题之间的依赖关系
 - **判断**：**最优子结构性质**
 - **求解**：先定最小子问题（初值），**自底向上**求解
- 实例：矩阵链相乘问题
 - **目标**：矩阵链乘法次数最少的加括号位置
 - 穷举法（二叉树求解，复杂度分析）
 - 动态规划算法的**递归**实现
 - 动态规划算法的**迭代**实现



矩阵连乘问题

问题： 给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$ ，其中 A_i 为 $P_{i-1} \times P_i$ 阶矩阵， $i=1, 2, \dots, n$ 。

试确定计算矩阵连乘积的计算次序，使得矩阵链相乘需要的**总次数最少**。

输入： 向量 $P=\langle P_0, P_1, \dots, P_n \rangle$ ，其中 $P_0, P_1, \dots, P_n$ 为 n 个矩阵的行数与列数。

输出： 矩阵链乘法次数最少的加括号位置。

矩阵连乘问题动态规划法

1. 分段：子问题划分

将矩阵连乘积 $A_i A_{i+1} \dots A_j$ 简记为 $A[i:j]$ ，这里 $i \leq j$

输入向量： $\langle P_{i-1}, P_i, \dots, P_j \rangle$

其最好划分的运算次数： $m[i, j]$

2. 分析：子问题的依赖关系

考察计算 $A[i:j]$ 的最优计算次序。设这个计算次序在矩阵 A_k 和 A_{k+1} 之间将矩阵链断开， $i \leq k < j$ ，即最优划分最后一次相乘发生在矩阵 k 的位置，则其相应完全加括号方式为 $(A_i A_{i+1} \dots A_k)(A_{k+1} A_{k+2} \dots A_j)$

矩阵连乘问题动态规划法

递推方程：

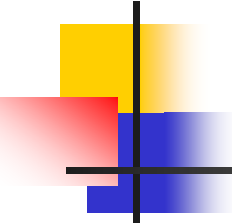
$m[i, j]$ ：得到 $A[i:j]$ 的最少的相乘次数。可递归定义为：

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \} & i < j \end{cases}$$

$$(A_i A_{i+1} \dots A_k) (A_{k+1} A_{k+2} \dots A_j)$$

$$P_{i-1} \times P_k \quad P_k \times P_j$$

该问题满足优化原则！



实例

- **输入：** $P = \langle 30, 35, 15, 5, 10, 20 \rangle$
 $n = 5$
- **矩阵链：** $A_1 A_2 A_3 A_4 A_5$ ，其中
 $A_1 : 30 \times 35, A_2 : 35 \times 15, A_3 : 15 \times 5,$
 $A_4 : 5 \times 10, A_5 : 10 \times 20$
- **备忘录：** 存储所有子问题的最小乘法**次数**
及得到这个值的划分**位置**

备忘录 $m[i, j]$

■ $P = \langle 30, 35, 15, 5, 10, 20 \rangle$

$r=1$	$m[1,1]=0$	$m[2,2]=0$	$m[3,3]=0$	$m[4,4]=0$	$m[5,5]=0$
$r=2$	$m[1,2]=15750$	$m[2,3]=2625$	$m[3,4]=750$	$m[4,5]=1000$	
$r=3$	$m[1,3]=7875$	$m[2,4]=4375$	$m[3,5]=2500$		
$r=4$	$m[1,4]=9375$	$m[2,5]=7125$			
$r=5$	$m[1,5]=11875$				

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\} & i < j \end{cases}$$

标记函数 $s[i, j]$

$r=2$	$s[1,2]=1$	$s[2,3]=2$	$s[3,4]=3$	$s[4,5]=4$
$r=3$	$s[1,3]=1$	$s[2,4]=3$	$s[3,5]=3$	
$r=4$	$s[1,4]=3$	$s[2,5]=3$		
$r=5$	$s[1,5]=3$			

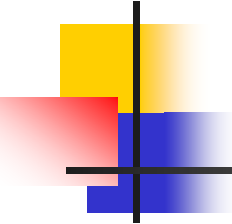
解的追踪: $s[1,5]=3 \rightarrow (A_1 A_2 A_3)(A_4 A_5)$

$s[1,3]=1 \rightarrow A_1(A_2 A_3)$

输出

计算顺序为: $(A_1(A_2 A_3))(A_4 A_5)$

最少的乘法次数: $m[1,5]=11875$



两种实现的比较

- 递归实现：时间复杂度高，空间开销小
- 迭代实现：**时间复杂度低，空间消耗多**
 - 原因：递归实现子问题多次重复计算，子问题计算次数呈指数增长。迭代实现每个子问题只计算一次
- 动态规划时间复杂度：
 - **备忘录各项计算量之和+追踪解工作量**
 - 追踪解的工作量通常是问题规模的多项式函数，一般不超过计算的工作量

组合问题中的动态规划法

最长公共子序列

子序列：若给定序列 $X=\{x_1, x_2, \dots, x_m\}$ ，则另一序列 $Z=\{z_1, z_2, \dots, z_k\}$ ，是 X 的子序列是指：存在一个**严格递增**下标序列 $\{i_1, i_2, \dots, i_k\}$ 使得对于所有 $j=1, 2, \dots, k$ 有： $z_j=x_{i_j}$ 。

例：序列 $Z=\{B, C, D, B\}$ 是 $X=\{A, B, C, B, D, A, B\}$ 的子序列，相应的递增下标序列为 $\{2, 3, 5, 7\}$ 。

给定两个序列 X 和 Y ，当另一序列 Z 既是 X 的子序列又是 Y 的子序列时，称 Z 是序列 X 和 Y 的**公共子序列**。

最长公共子序列

- **问题：** 给定两个序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ ，找出 X 和 Y 的最长公共子序列 (Longest Common Subsequence, LCS)

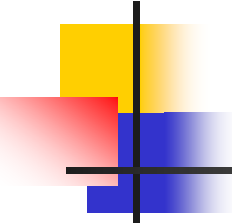
- **实例：**

X: A B C B D A B

Y: B D C A B A

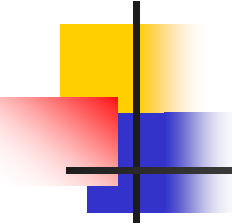
最长公共子序列: B C B A, 长度4

- ➔ 不是唯一的，长度相等情况下，可能有多不同公共子序列，只需给出一个即可



穷举法

- 不妨设 $m \leq n, |X|=m, |Y|=n$
- 算法：依次检查X的每个子序列在Y中是否出现



穷举法

- 不妨设 $m \leq n, |X|=m, |Y|=n$
- 算法：依次检查X的每个子序列在Y中是否出现
- 时间复杂度：
 - X有 2^m 个子序列
 - 每个子序列 $O(n)$ 时间

最坏情况下时间复杂度： $O(n2^m)$

子问题界定

- X的终止位置是 i , Y的终止位置是 j
- $X_i = \{x_1, x_2, \dots, x_i\}$, $Y_j = \{y_1, y_2, \dots, y_j\}$
- 参数 i 和 j 界定子问题

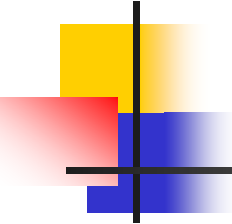


子问题间的依赖关系

- 假设两个序列 $X=\{x_1, x_2, \dots, x_m\}$, $Y=\{y_1, y_2, \dots, y_n\}$, $Z=\{z_1, z_2, \dots, z_k\}$ 为 X 和 Y 的 LCS, 那么
 - 若 $x_m = y_n \rightarrow z_k = x_m = y_n$,
且 Z_{k-1} 是 X_{m-1} 与 Y_{n-1} 的 LCS
 - 若 $x_m \neq y_n$, $z_k \neq x_m \rightarrow$
 Z 是 X_{m-1} 与 Y 的 LCS
 - 若 $x_m \neq y_n$, $z_k \neq y_n \rightarrow$
 Z 是 X 与 Y_{n-1} 的 LCS



满足最优子结构性质和子问题重叠性



优化函数的递推方程

- 令X和Y的子序列
- $X_i = \{x_1, x_2, \dots, x_i\}$, $Y_j = \{y_1, y_2, \dots, y_j\}$
- $C[i, j]$: X_i 与 Y_j 的LCS的长度

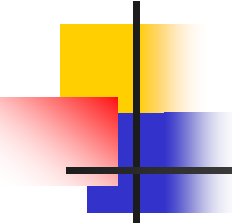
$$C[i, j] = \begin{cases} 0 & \text{若 } i=0 \text{ 或 } j=0 \\ C[i-1, j-1]+1 & \text{若 } i, j > 0, x_i = y_j \\ \max \{C[i, j-1], C[i-1, j]\} & \text{若 } i, j > 0, x_i \neq y_j \end{cases}$$

-

算法的伪码

■ 算法 LCS(X, Y, m, n)

1. for $i \leftarrow 1$ to m do $C[i, 0] \leftarrow 0$ 
2. for $j \leftarrow 1$ to n do $C[0, j] \leftarrow 0$
3. for $i \leftarrow 1$ to m do
4. for $j \leftarrow 1$ to n do 
5. if $X[i] = Y[j]$
6. then $C[i, j] \leftarrow C[i-1, j-1] + 1$
7. $B[i, j] \leftarrow \text{“}\nwarrow\text{”}$ 
8. else if $C[i-1, j] \geq C[i, j-1]$
9. then $C[i, j] \leftarrow C[i-1, j]$
10. $B[i, j] \leftarrow \text{“}\uparrow\text{”}$
11. else $C[i, j] \leftarrow C[i, j-1]$
12. $B[i, j] \leftarrow \text{“}\leftarrow\text{”}$



追踪解

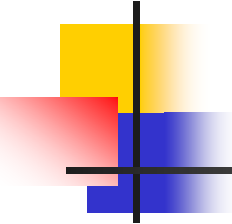
- 算法StructureSequence(B, i, j)
 - 输入: $B[i, j]$
 - 输出: X 与 Y 的最长公共子序列
 1. if $i=0$ or $j=0$ then return //序列为空
 2. if $B[i, j] = \text{“}\nwarrow\text{”}$
 3. then 输出 $X[i]$
 4. StructureSequence($B, i-1, j-1$)
 5. else if $B[i, j] = \text{“}\uparrow\text{”}$
 6. then StructureSequence($B, i-1, j$)
 7. else StructureSequence($B, i, j-1$)

标记函数的实例

- 输入: $X = \langle A, B, C, B, D, A, B \rangle$
 $Y = \langle B, D, C, A, B, A \rangle$

X \ Y	1	2	3	4	5	6
1	B[1,1]=↑	B[1,2]=↑	B[1,3]=↑	B[1,4]=↖	B[1,5]=←	B[1,6]=↖
2	B[2,1]=↖	B[2,2]=←	B[2,3]=←	B[2,4]=↑	B[2,5]=↖	B[2,6]=←
3	B[3,1]=↑	B[3,2]=↑	B[3,3]=↖	B[3,4]=←	B[3,5]=↑	B[3,6]=↑
4	B[4,1]=↑	B[4,2]=↑	B[4,3]=↑	B[4,4]=↑	B[4,5]=↖	B[4,6]=←
5	B[5,1]=↑	B[5,2]=↑	B[5,3]=↑	B[5,4]=↑	B[5,5]=↑	B[5,6]=↑
6	B[6,1]=↑	B[6,2]=↑	B[6,3]=↑	B[6,4]=↖	B[6,5]=↑	B[6,6]=↖
7	B[7,1]=↑	B[7,2]=↑	B[7,3]=↑	B[7,4]=↑	B[7,5]=↑	B[7,6]=↑

解: $X[2], X[3], X[4], X[6]$, 即 B, C, B, A

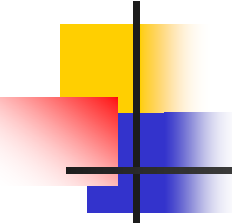


算法的时空复杂度

- 计算优化函数和标记函数
 - 赋初值，为 $O(m)+O(n)$
 - 计算优化、标记函数迭代次 $\Theta(mn)$ ，循环体内常数运算
 - ∴ 总的时间复杂度为 $\Theta(mn)$
- 构造解（追踪）
 - 每步缩小X或Y的长度，时间 $\Theta(m+n)$

算法时间复杂度： $\Theta(mn)$

空间复杂度： $\Theta(mn)$



小结

- 最长公共子序列问题的建模
- 子问题边界的界定
- 递推方程及初值，优化原则判定
- 伪码
- 标记函数与解的追踪
- 时空复杂度

背包问题(Knapsack Problem)

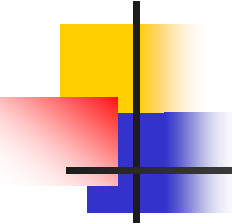
- 一个旅行者随身携带一个背包。可以放入背包的物品有 n 种，每种物品的重量和价值分别为 w_i v_i 。如果背包的最大承重限制是 b ，每种物品可以放多个。怎么样选择放入背包的物品使得背包所装物品**价值最大**？
- 不妨设上述 w_i v_i b 都是正整数。

实例： $n=4$, $b=10$

$$v_1=1, v_2=3, v_3=5, v_4=9$$

$$w_1=2, w_2=3, w_3=4, w_4=7$$



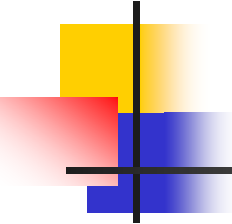


建模

- 解是 $\langle x_1, x_2, \dots, x_n \rangle$ ，其中 x_i 是装入背包的第 i 种物品个数

目标函数

约束条件



建模

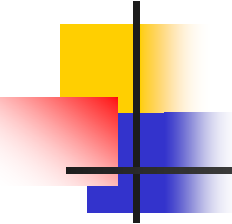
- 解是 $\langle x_1, x_2, \dots, x_n \rangle$ ，其中 x_i 是装入背包的第 i 种物品个数

目标函数 $\max \sum_{i=1}^n v_i x_i$

约束条件 $\sum_{i=1}^n w_i x_i \leq b, \quad x_i \in N$

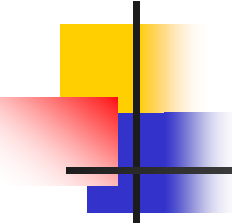
线性规划问题：由线性条件约束的线性函数取最大或最小的问题
Linear Programming

整数规划问题：线性规划问题的变量 x_i 都是非负整数



子问题界定和计算顺序

- **子问题界定：** 由参数 k 和 y 界定
 - k ：考虑物品 $1, 2, \dots, k$ 的选择
 - y ：背包总重量不超过 y
- 原始输入： $k=n, y=b$
- **子问题计算顺序：**
 - $k=1, 2, \dots, n$
 - 对于给定的 k , $y=1, 2, \dots, b$



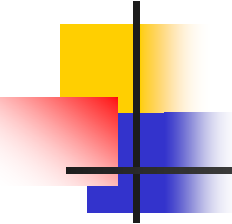
优化函数的递推方程

$F_k(y)$: 装前 k 种物品, 总重不超过 y
背包达到的最大价值

$$F_k(y) = \max\{F_{k-1}(y), F_k(y-w_k)+v_k\}$$

$$F_0(y) = 0, 0 \leq y \leq b, F_k(0) = 0, 0 \leq k \leq n$$

$$F_1(y) = \left\lfloor \frac{y}{w_1} \right\rfloor v_1, \quad F_k(y) = -\infty \quad y < 0$$



标记函数

$i_k(y)$: 装前 k 种物品, 总重不超过 y , 背包达到最大价值时装入物品的**最大标号**

$$i_k(y) = \begin{cases} i_{k-1}(y) & F_{k-1}(y) > F_k(y - w_k) + v_k \\ k & F_{k-1}(y) \leq F_k(y - w_k) + v_k \end{cases}$$

$$i_1(y) = \begin{cases} 0 & y < w_1 \\ 1 & y \geq w_1 \end{cases}$$

动态规划求解背包问题

输入: $n=4, b=10$

$v_1=1, v_2=3, v_3=5, v_4=9$

$w_1=2, w_2=3, w_3=4, w_4=7$



$F_k(y)$ 和 $i_k(y)$ 的计算表如下:

$k \backslash y$	1	2	3	4	5	6	7	8	9	10
1										
2										
3										
4										

动态规划求解背包问题

输入: $n=4, b=10$

$$v_1=1, v_2=3, v_3=5, v_4=9$$

$$w_1=2, w_2=3, w_3=4, w_4=7$$



$F_k(y)$ 的计算结果如下:

$k \backslash y$	1	2	3	4	5	6	7	8	9	10
1	0	1	1	2	2	3	3	4	4	5
2	0	1	3	3	4	6	6	7	9	9
3	0	1	3	5	5	6	8	10	10	11
4	0	1	3	5	5	6	9	10	10	12

输入: $n=4, b=10$

$v_1=1, v_2=3, v_3=5, v_4=9$

$w_1=2, w_2=3, w_3=4, w_4=7$

追踪解

$i_k(y)=$

$k \backslash y$	1	2	3	4	5	6	7	8	9	10
1	0	1	1	1	1	1	1	1	1	1
2	0	1	2	2	2	2	2	2	2	2
3	0	1	2	3	3	3	3	3	3	3
4	0	1	2	3	3	3	4	3	4	4

$$i_4(10)=4 \Rightarrow x_4 \geq 1$$

$$i_4(10 - w_4) = i_4(3) = 2 \Rightarrow x_2 \geq 1, \quad x_4 = 1, \quad x_3 = 0$$

$$i_2(3 - w_2) = i_2(0) = 0 \Rightarrow x_2 = 1, \quad x_1 = 0$$

解: $x_1=0, x_2=1, x_3=0, x_4=1$, 价值12

追踪算法

■ 算法 TrackSolution

- 输入: $i_k(y)$ 表, $k=1, 2, \dots, n$, $y=1, 2, \dots, b$
- 输出: $x_1, x_2, \dots, x_n$, n 种物品的装入量

1. **for** $j \leftarrow 1$ **to** n **do** $x_j \leftarrow 0$

2. $y \leftarrow b$, $j \leftarrow n$

3. $j \leftarrow i_j(y)$

初始追踪位置

4. $x_j \leftarrow 1$

5. $y \leftarrow y - w_j$

6. **while** $i_j(y)=j$ **do**

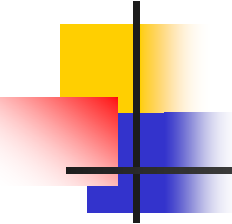
继续放第 j 种物品

7. $y \leftarrow y - w_j$

8. $x_j \leftarrow x_j + 1$

9. **if** $i_j(y) \neq 0$ **then goto** 3.

继续追踪下一种



时间复杂度

根据公式

$$F_k(y) = \max\{F_{k-1}(y), F_k(y-w_k) + v_k\}$$

备忘录需计算 nb 项，每项常数时间，计算时间为 $O(nb)$

伪多项式时间算法：时间为参数 b 和 n 的多项式，不是输入规模的多项式。参数 b 是整数，表达 b 需要 $\log b$ 位，输入规模以 n 和 $\log b$ 为参数。